

Hybrid architecture, and closing the loop

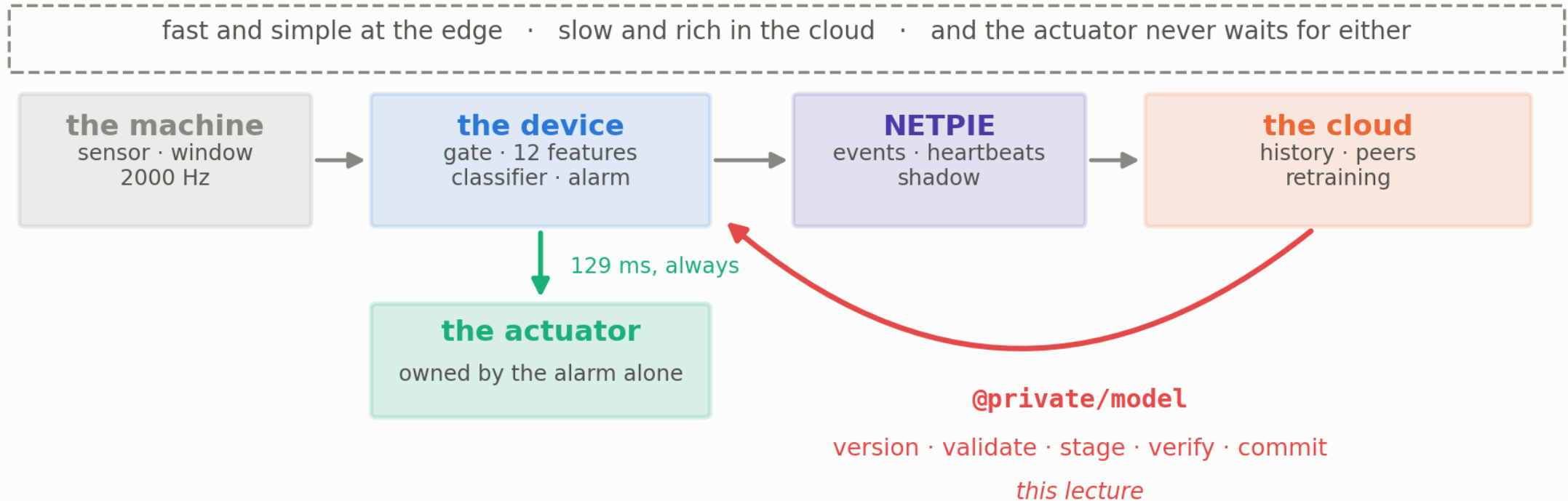
The only channel in this course that can make the machine less safe

- The reference architecture end to end, and the one rule that holds it together
- The latency budget: what round trip a cloud-in-the-loop design would have to fit
- The update message, and the apply path — validate, stage, verify, commit, ack
- 19 downlink messages against three apply paths, and what the machine is left with
- Which artefact is dangerous to update — and it is not the big one
- When the sensor lies: four ordinary faults, and the gate that costs almost nothing

Notebook: `iaiiot26_Lect13_nb.ipynb`

The whole system, finally

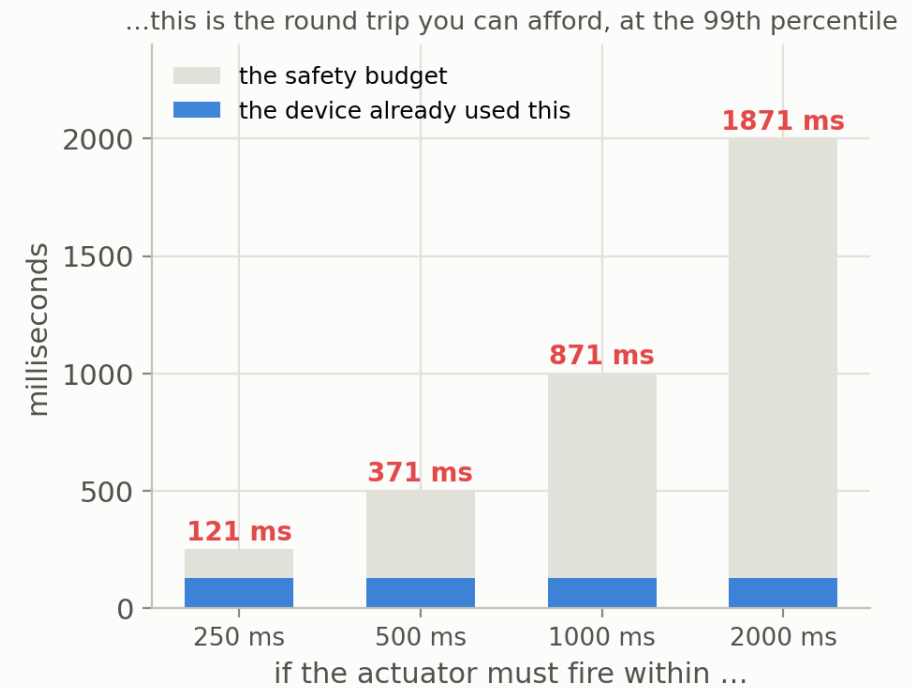
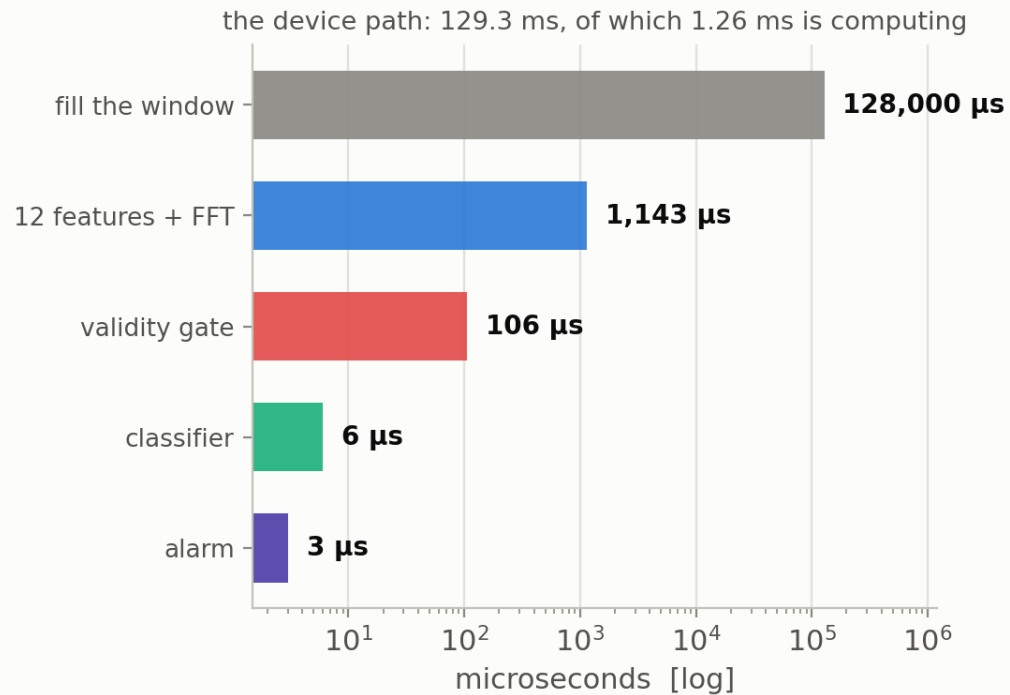
- Fast and simple at the edge, slow and rich in the cloud — and the actuator never waits for either.



Every arrow but one adds information. The downlink is the only one that points into the safety path, and the only one that can take protection away.

How much loop is there?

- Work out the budget before deciding what may be inside it. The window dominates; all the computing is about one per cent of it.



*You cannot know whether the cloud fits inside a control loop until you have measured the TAIL of your own round trip.
The mean is not the number that matters; the 99th percentile is.*

The device has already spent 129 ms. What is left is what a cloud round trip must fit inside — at the 99th percentile, not the mean.

The update message, and the apply path

- A data message that gets lost costs one window of history. An update message that is wrong costs the machine. The schema and the sequence both reflect that.

the update message

v	schema version	1
kind	which artefact	"alarm"
ver	monotonic; never goes back	4
t / ttl	issued at, and shelf life	1789000000 / 30 d
sum	sha256 of the payload	"9f2c..."
by	who approved it	"V. Toochinda"
payload	25 numbers, or 39	{ ... }

*the checksum catches corruption, not malice —
and the device logs which one it is relying on*

the apply path

1 · validate

structure, version, checksum,
freshness, sane numbers

2 · stage

into the standby slot.
Nothing running is touched

3 · verify

run it in shadow on windows
the device kept

4 · commit

one pointer. A power cut leaves
the old spec, not half of two

5 · ack

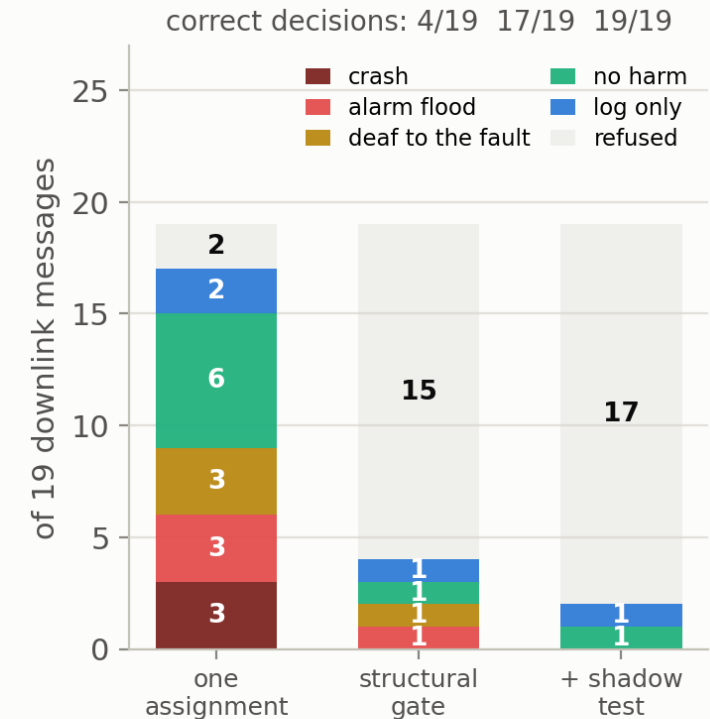
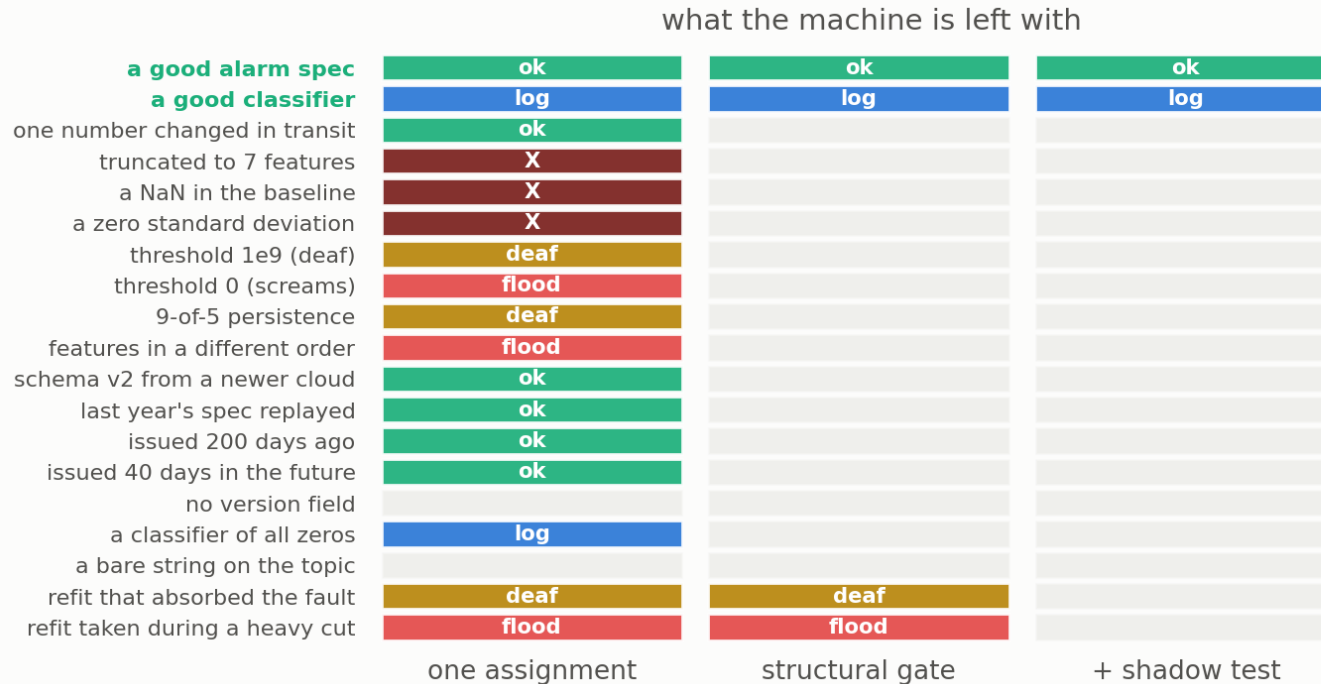
version, decision and reason
back up the link

Any of the five can refuse, and refusing is not failure — the device keeps running the spec it already had.

Stage into a standby slot, verify by running it, then commit with one pointer. A power cut mid-update leaves the old spec — never half of two.

What a hostile update does to a device

- 19 messages arrive on the downlink topic, two of them legitimate. The measurement is not "did it parse" but what the machine is left with.



Two of the 19 are legitimate and must be accepted. A gate that refuses everything is not a gate, it is a disconnected wire.

A one-line apply is right 4 times in 19 and leaves the machine unprotected in 9: 3 crash, 3 go deaf to a real spall, 3 flood the plant.

What a structural check cannot see

refit that absorbed the fault

Twelve finite means, twelve positive standard deviations, a threshold of 4.01, a fresh timestamp and a correct checksum.

It has learned that a spalled bearing is normal. It will never raise again.

refit taken during a heavy cut

Equally well formed, equally fresh, equally correct on every structural test.

It has learned that a loaded machine is the baseline. It raises on everything.

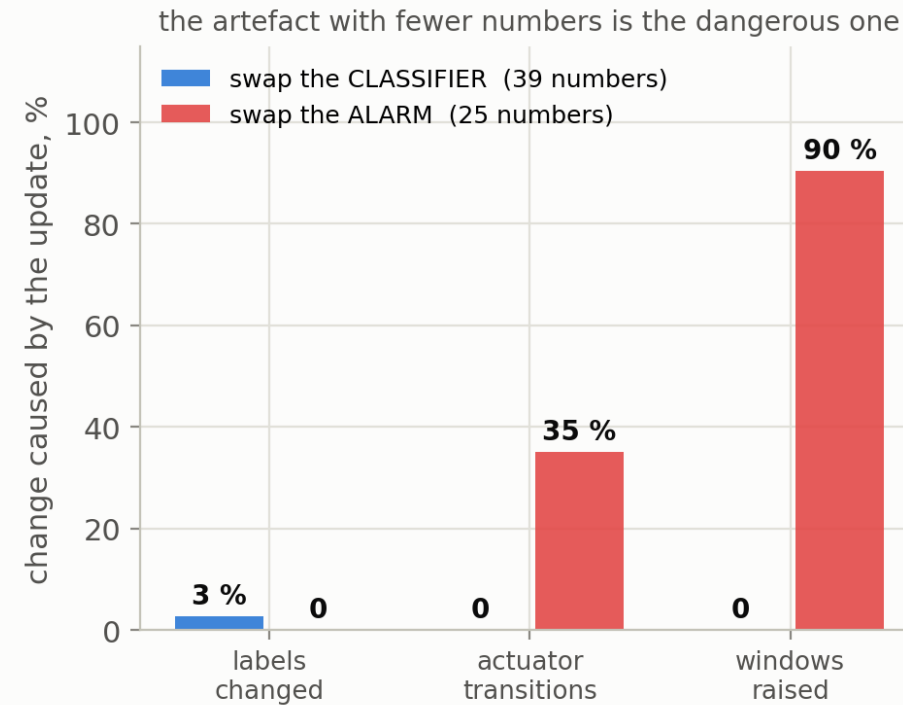
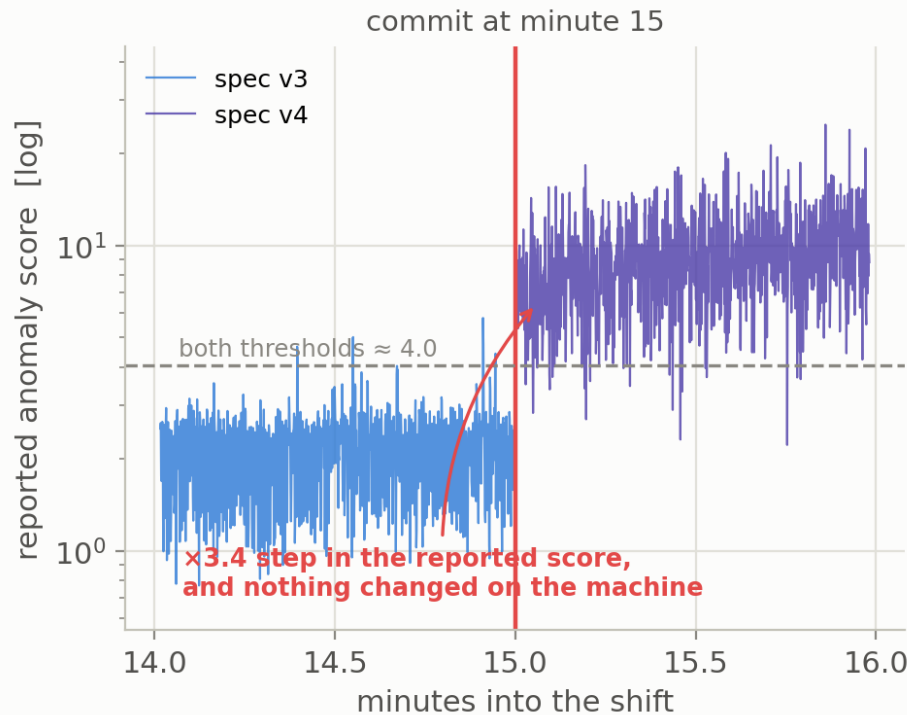
```
def verify(candidate):
    q = rate(candidate, QUIET)      # windows from a stretch nothing happened in
    if q > FA_MAX:
        return False, 'would flag %d%% of a quiet stretch' % (100 * q)

    a = rate(candidate, ALARMED)    # windows that made THIS device raise
    if a < RECALL_MIN:
        return False, 'deaf: flags only %d%% of retained alarms' % (100 * a)
    return True, 'ok'
```

Where does a device get windows that made it raise? It kept them — Lecture 12's "on alarm + context" bursts, for a reason nobody stated at the time.

Which artefact is dangerous to update

- The classifier is 39 numbers, the alarm is 25. One of them needs a shadow test and a named approver — and students guess wrong.

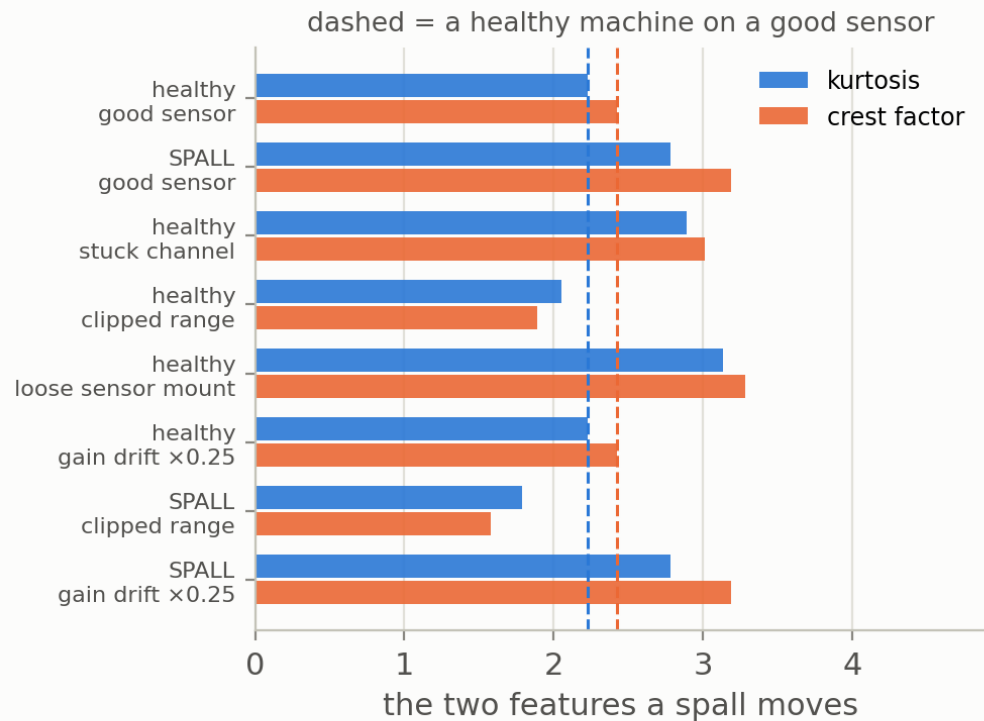


The classifier drives the log. The alarm drives the actuator. Gate the update in proportion to what it can break, not to how big it is.

Carry the spec version in the uplink, or two years of trend has a step in it that nobody can explain — six bytes against the whole history.

When the sensor lies

- Every model in this course assumes the accelerometer works, and none of them checks. Four ordinary faults: stuck, clipped, loose bolt, gain drift.



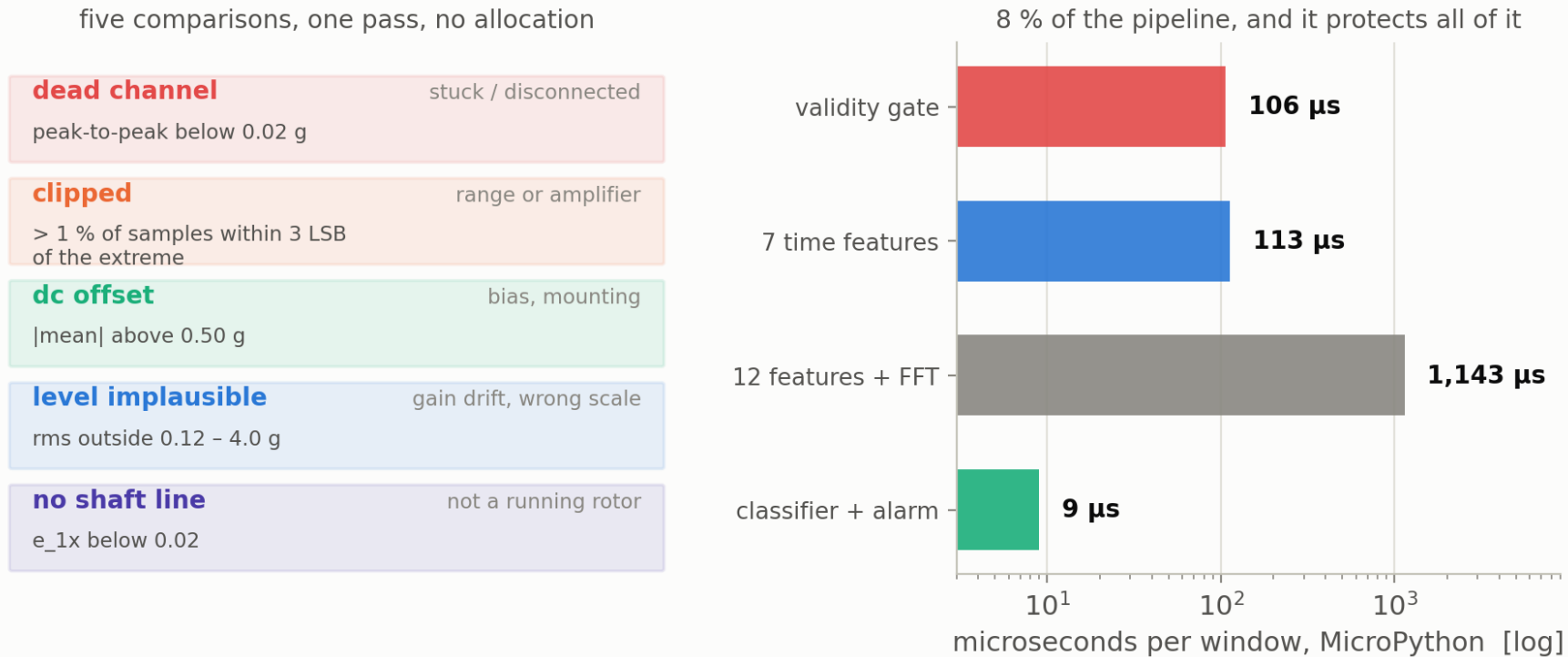
"normal" (1.4)	passes
"bearing" (1.3)	passes
"normal" (9.8)	dead channel
"normal" (1.7)	clipped
"bearing" (1.5)	passes
"normal" (3.0)	level implausible
"normal" (1.8)	clipped
"normal" (0.8)	level implausible
what the model said	what the gate said

A clipped sensor made a spalled bearing look HEALTHIER than a healthy machine — kurtosis 1.79 against 2.23 — and the alarm never raised.

Noise makes a model uncertain. A sensor fault makes it confident and wrong — which is why "it will degrade gracefully" is not true here.

Five comparisons, before any feature

- The fix is not a better model. It is one pass over the raw window asking whether this is plausibly a working accelerometer on this machine.



False rejections on 30 runs of healthy, imbalance and bearing machines with a good sensor: 0.0000. The gate costs nothing it does not have to.

5 of 6 caught, false rejections 0.0000 over 30 good runs, 106 μs. It misses the loose bolt — and that one needs a human.

Degraded modes, and the log

- Not one row says “stop deciding”. A device that stops protecting the machine when the network does is a sensor with extra steps.

what fails	what the device does	protection
the network drops	keep deciding, queue the report	none
the broker refuses	keep deciding, back off, retry	none
an update fails validation	keep the running spec, ack the reason	none
the committed spec misbehaves	roll back to last_good, tell the cloud	none
the classifier is uncertain	label "uncertain"; the alarm is unaffected	none
the sensor fails the gate	hold the actuator, publish sensor-fault	DEGRADED
features raise	hold the actuator, log the window	DEGRADED
the spec has expired	keep using it, flag it, escalate	DEGRADED

Not one row says "stop deciding". A device that stops protecting the machine when the network does is not an edge device.

Three rows are DEGRADED: the device is still safe — it holds its last state — but it is no longer protecting, and somebody has to know. "Hold and be quiet about it" is never acceptable.

Lab 13 — closing the loop

A · the downlink

Add `net_apply.py`. Push a new alarm spec from Colab; watch it validate, verify, commit and ack.

Then push three broken ones and record which gate caught each.

B · pull the plug

With the new spec running, disconnect WiFi in Wokwi.

Prove the device still detects and still actuates. Then reconnect and show the queued reports arrive.

C · break the sensor

Add `validity.py`. Inject a stuck channel and a clipped range and record what the model said before and after the gate.

Then defend one limit you changed.

```
# downlink: {'v':1,'kind':'alarm','ver':4,'ok':0,
#           'why':'shadow: deaf: flags only 1% of retained alarms',
#           'running':{'alarm':3,'classifier':3}}

win,label,margin,score,driver,raised,gate,us
41,bearing,2.31,4.08,kurt,1,ok,1243
42,-,0.00,0.00,-,1,clipped,118      <- held, not guessed
```

Deliverable: the ack for every update you pushed, a video or log of the device running with the WiFi off, and one defended change to a gate limit.

Everything that can refuse, must be able to



1. Work out the latency budget first. The device uses 129 ms of it; whatever is left is what a round trip must fit inside.
2. Validate, stage, verify, commit, ack — in that order. A one-line apply was right 4 times in 19.
3. Verify by RUNNING the candidate — on quiet windows and on windows that made this device raise. One without the other is half a check.
4. Gate in proportion to what an update can break: the alarm's 25 numbers, not the classifier's 39.
5. Gate the sensor before the model. 106 μ s caught 5 of 6 faults at a false-rejection rate of 0.000.
6. Never stop deciding. Degrade, hold, log, escalate — and never hold quietly.